

WHAM-XREX-PFMG474 — LabVIEW Interface Command Reference

This document is the complete serial-command contract a LabVIEW programmer needs to build a control front-end for the **WHAM-XREX-PFMG474** (the closed-loop PID controller driving four Transrex ISR-2126 magnet power supplies). It covers:

1. 1
2. 2
3. 3
4. 4
5. 5
6. 6

Everything here is the *current* wire protocol. Channel numbers are **1-based** everywhere. All commands and replies are plain ASCII text.

1 1. Serial link and framing (must-read)

Parameter	Value
Baud rate	115200
Data bits / parity / stop	8 / none / 1 (8N1)
Flow control	none
Line terminator (TX and RX)	\r\n (carriage return + line feed)
Character encoding	ASCII

Framing invariant — one line per packet. Every command is one line, and every reply/event is one line terminated by **\r\n**. There are no length prefixes, no binary framing, no multi-line reassembly. In LabVIEW:

- write a command by sending **COMMAND... \r\n** (e.g. **STATE? \r\n**);
- read a reply by reading **whole lines** (VISA Read until you see **\r\n**).

Two kinds of line come back:

OK ... a command succeeded (optional data after "OK ")
 ERR <code> <msg> a command failed (<code> is a stable integer, see §4)
 !EVT ... an UNSOLICITED telemetry event (see §6) - not a reply

Because !EVT lines can arrive at any time and are interleaved between OK/ ERR replies, the LabVIEW read loop must **classify each line by its first token** (OK / ERR / !EVT) and dispatch accordingly. A reply to the command you just sent may be preceded by zero or more !EVT lines.

Mnemonic matching. Commands are SCPI-style and **case-insensitive**; each token also has a short form (the leading uppercase run). To keep the LabVIEW code unambiguous, **always send the full form exactly as written in this document** (e.g. SHOT:START, SOURce:SETpoint).

Channel numbering. Channels are **1-based** on the wire, 1..N where N = 4 on this build (confirm with CONFIG:CHANnels?, see §3).

2. Shot configuration — write commands

These commands set up and start a shot. Every one returns OK on success, or ERR <code> <msg> on failure.

2.1 State machine (arming)

Command	Effect	Reply
ARM	IDLE → ARMED. Must be sent before SHOT:START or SOURce:RUN. Refuses if a fault is latched or the interlocks aren't satisfied.	OK
DISARM	ARMED → IDLE (stand down without firing).	OK

2.2 2.2 Per-channel setup

Command	Effect	Reply
SOURce:ENABle <ch> <0 1>	Enable (1) / disable (0) the PFM output for channel ch . 0 = no waveform at all.	OK
SOURce:SETPoint <ch> <hz>	Set channel ch 's target output frequency (Hz). Clamped into 3000–150000 Hz. Write-only — read it back via SOURce:STATus?.	OK
PID:LOOPMODE <ch> <0 1>	Per-channel loop mode: 0 = open-loop (no feedback correction), 1 = closed-loop (default). ch may be 0 to set all channels at once.	OK
PID:GAINS <ch> <kp> <ki> <kd>	Set channel ch 's PID gains (closed-loop only). Resets that channel's integrator.	OK
CHANnel:NICKname <ch> <name>	Optional human label for channel ch (≤ 15 chars, no spaces).	OK
XREX:CHANnel:ENAOut <ch> <0 1>	Drive the channel's ENA_OUT fiber output (high = energize).	OK
XREX:CHANnel:CONtactOut <ch> <0 1>	Drive the channel's CONTACT_OUT fiber output.	OK

The fiber outputs (XREX:CHANnel:ENAOut / CONtactOut) are the physical "ready to energize" lines to each Transrex. ARM refuses unless every **enabled** channel's ENA_OUT and CONTACT_OUT are both 1, so set these before arming.

2.3 2.3 Shot profile (the bounded shot — primary use)

Command	Effect	Reply
SHOT:TIMing <rampUpS> <flatTopS> <rampDownS>	Set the shared shot timing (seconds, applied to all channels): ramp-up time, flat-top time, ramp-down time. Must be > 0 .	OK
SHOT:CURREnt <ch> <demandA>	Set channel ch 's peak demand current (amps) for the next shot. Clamped into 0..6000 A per channel.	OK

2.4 2.4 Starting / stopping output

Command	Effect	Reply
SHOT:START	Start the profiled shot on every enabled channel. Requires ARMED , and SHOT:TIMing to have been set. Ramps up → flat-top → ramps down, then auto-stops back to IDLE .	OK
SOURce:RUN	Start continuous closed-loop output on every channel. Requires ARMED . No auto-stop — it runs until SOURce:STOP or a fault.	OK
SOURce:STOP	Stop all output immediately and return to IDLE (also aborts any shot/ramp in progress).	OK
SOURce:RAMP <ch> <startHz> <endHz> <durMs>	One-shot linear setpoint ramp on channel ch (bench/trajectory use; independent of the shot profile).	OK

Use **SHOT:START** (not **SOURce:RUN**) for normal operation. **SHOT:START** runs a bounded, timed profile and stops itself; **SOURce:RUN** is an open-ended continuous loop with no shot clock.

2.5 2.5 Waveform logging (time-series capture)

Command	Effect	Reply
LOG:ARM <ch> <maxSamples> <decim>	Arm waveform logging. ch = channel to log (0 = log all channels simultaneously); maxSamples = 1..1750; decim = 1.. (sample every Nth control tick).	OK

Log data is pulled afterward with **LOG:DATA?** (§3). The log records {setpoint, measured, output} (all in Hz) per sample.

2.6 2.6 Interlocks and fault clear

Command	Effect	Reply
EXTErnal:ENABle <0 1>	Turn the external-enable interlock (PF13 input) on/off. When ON, ARM and SHOT:START require PF13 to read HIGH.	OK
EXTErnal:TRIGger <0 1>	Turn the external-trigger feature on/off (a rising edge on PF15 fires a shot while ARMED).	OK
FAULT:CLEAR	Clear a latched fault. Does not itself re-enable output — that's the next ARM + SHOT:START.	OK

3 3. Status / configured-value / fault polling — read commands

These end in ? (or are STATE?/*IDN?) and return a value to read back into LabVIEW controls.

3.1 3.1 Identity and system

Command	Reply	Meaning
*IDN?	OK WHAM-XREX-PFMG474 REVA v0.1 <github>	Board + firmware identity. Check this at connect.
CONFIg:CHANnels?	OK 4	Number of output channels (N).
STATE?	see §3.3	Current operating state + fault detail.
SYS:TIME?	OK <ms>	Milliseconds since boot (monotonic).
SYS:TELEM?	OK 4	Telemetry schema version (§6).

3.2 3.2 Configured values and live status (per channel)

Command	Reply	Meaning
SOURce:STATUs? <ch>	OK <running> <setpointHz> <measuredHz> <outputHz>	running 0/1; then the target, measured (feedback), and actual output frequencies in Hz.
SOURce:ENABle? <ch>	OK <0 1>	Whether the channel's output is enabled.
PID:GAINS? <ch>	OK <kp> <ki> <kd>	Current PID gains (floats).
PID:LOOPMODE? <ch>	OK <0 1>	Open (0) / closed (1) loop.
SHOT:TIMing?	OK <rampUpS> <flatTopS> <rampDownS>	Shared shot timing (seconds). ERR 12 if never set.
SHOT:CURREnt? <ch>	OK <demandA>	Channel's peak demand current for the next shot.
CHANnel:NICKname? <ch>	OK <name> (or OK -)	The channel's nickname (- = none).
XREX:CHANnel:ENAOut? <ch>	OK <0 1>	ENA_OUT fiber output state.
XREX:CHANnel:CONtactOut? <ch>	OK <0 1>	CONTACT_OUT fiber output state.
EXTErnal:ENABle?	OK <0 1>	External-enable interlock on/off.
EXTErnal:INPut?	OK <0 1>	Raw PF13 level (interlock input).
EXTErnal:TRIGger?	OK <0 1>	External-trigger feature on/off.
EXTErnal:TRIGger:INPut?	OK <0 1>	Raw PF15 level (trigger input).

There is no `SOURCE:SETpoint?` — setpoint is write-only. Read the target via `SOURCE:STATUS?` (the `setpointHz` field).

3.3 STATE? — the authoritative state + fault word

`STATE?` returns one of:

```
OK IDLE
OK ARMED
OK FIRING
OK FAULT GENERAL
OK FAULT OVERCURRENT <ch>
OK FAULT EXTERNAL_ENABLE
OK FAULT ENABLE_OUTPUT <ch>
OK FAULT ENERPRO <ch>
```

`<ch>` is 1-based and present only for the per-channel fault types. Poll this (cheaply) in the LabVIEW loop to drive the front-end's state indicator and fault lamp.

3.4 Fault / health polling (raw and interpreted)

Command	Reply	Meaning
<code>FAULT?</code>	OK 0 or OK 1	Any fault latched (either source). 1 = output was safed.
<code>XREX:CHANnel:STATus? <ch></code>	OK WATER=HIGH LOW TMP=HIGH LOW ENERPRO=HIGH LOW OCP=HIGH LOW	Raw per-channel fault-pin levels (polarity-agnostic — HIGH/LOW is the raw pin state, NOT "faulted/ok").
<code>GDS?</code>	OK 01=HIGH 02=LOW ... 12=...	Raw snapshot of the 12 gate-driver-status pins (PE0–PE11). Diagnostic only.

Fault semantics for the front-end. A fault is **latched** until `FAULT:CLEAR` is sent, even after the physical condition clears. The five fault types are:

Type (from <code>STATE?</code>)	Channel?	Meaning
GENERAL	no	PC10/HRTIM hardware fault OR a gate-driver-status fault.
OVERCURRENT <ch>	yes	Per-channel overcurrent.
EXTERNAL_ENABLE	no	The PF13 external-enable interlock was lost while firing.
ENABLE_OUTPUT <ch>	yes	This channel's commanded <code>ENA_OUT/CONTACT_OUT</code> dropped while armed.
ENERPRO <ch>	yes	Per-channel Enerpro fault (reclassified from GENERAL).

On any fault: output ramps down and the controller settles in `FAULT`. Recovery is always `FAULT:CLEAR` → (re-assert `XREX:...:ENAOut/CONtactOut` if needed) → `ARM` → `SHOT:START`.

3.5 3.5 Waveform log pull

Command	Reply	Meaning
LOG:DATA? [ch]	OK <count> <rateHz> s1 m1 o1 s2 m2 o2 ...	<count> samples, captured at <rateHz>; then count triples of setpoint measured output (Hz). Omit [ch] when a single channel was armed; pass <ch> under all-channels mode.

LOG:DATA? is a **single line** but can be several KB long for a full log — size the VISA read buffer accordingly (or read incrementally until \r\n).

4 4. Error codes

Replies are ERR <code> <msg>. Codes are **stable** — parse the integer and map it:

Code	Meaning
1	Unknown command / mnemonic
2	Not uploading a table (send TABLE:BEGIN first — legacy path only)
3	Table full (legacy)
4	Invalid TABLE:STEP args (legacy)
5	Table empty (legacy FIRE)
6	Fault latched — clear with FAULT:CLEAR
7	QSPI test failed (diagnostic)
8	Invalid PFM input channel (diagnostic)
9	PFMIN:CAPTURE count out of range (diagnostic)
10	Carrier above PFM_MAX_CARRIER_FREQ_HZ (legacy)
11	Invalid channel — must be 1..N
12	Invalid arguments (wrong count / non-numeric / out of range)
13	Invalid state transition — e.g. SHOT:START while not ARMED
14	Invalid nickname (length / spaces / reserved -)
15	SHOT:START/ARM refused — EXTERNAL:ENABLE is ON and PF13 reads LOW
16	(retired — do not use)

The LabVIEW front-end should surface ERR 6, ERR 13, and ERR 15 as distinct operator messages.

5 5. Worked end-to-end shot sequence

Typical closed-loop profiled shot (channel 1, 3000 A demand, 5 s ramp-up, 2 s flat-top, 5 s ramp-down):

CONFig:CHANnels?	-> OK 4	(confirm channel count)
EXtErnal:ENABle 0	-> OK	(leave interlock off if unused)
XREX:CHANnel:ENAOut 1 1	-> OK	(energize fiber outputs)
XREX:CHANnel:CONtActOut 1 1	-> OK	
SOURce:ENABle 1 1	-> OK	(enable channel 1 output)
PID:LOOPMODE 1 1	-> OK	(closed loop)
PID:GAINS 1 <kp> <ki> <kd>	-> OK	(gains)
SHOT:CURRent 1 3000	-> OK	(demand current)
SHOT:TIMing 5 2 5	-> OK	(rampUp flatTop rampDown, seconds)

```

LOG:ARM 1 1750 1          -> OK          (optional: log every tick)
ARM                        -> OK          (IDLE -> ARMED)
SHOT:START               -> OK          (fire the shot)
    ... poll STATE? until it returns OK IDLE (shot auto-completes) ...
LOG:DATA? 1              -> OK 1750 <rate> ... (pull the waveform)
SOURCE:ENABle 1 0        -> OK          (shut channel down)
DISARM                   -> OK          (if still armed)

```

If anything returns ERR 6, do FAULT:CLEAR and re-arm. If ARM or SHOT:START returns ERR 15, either turn EXTERNAL:ENABle 0 or assert PF13 HIGH.

6 6. EXTRA CREDIT — receiving & decoding telemetry (!EVT)

The controller pushes **unsolicited, timestamped events** on state transitions and faults. The host doesn't have to poll for them.

6.1 6.1 Enabling and schema

Command	Reply	Meaning
SYS:EVENT 1	OK	Enable the live !EVT stream (default is already ON).
SYS:EVENT 0	OK	Silence the live stream (the flight recorder still records).
SYS:EVENT?	OK <0 1>	Current gate state.
SYS:TELEM?	OK 4	Schema version — re-verify on connect.
SYS:TIME?	OK <ms>	Monotonic clock the !EVT ticks are drawn from.
SYS:EVLOG?	OK <n> then n lines of !EVT ...	Replay the retained flight-recorder history (oldest first).

6.2 6.2 Packet grammar

An event is one line:

```

!EVT <tick_ms> STATE <IDLE|ARMED|FIRING|FAULT>
!EVT <tick_ms> FAULT <GENERAL|OVERCURRENT|EXTERNAL_ENABLE|ENABLE_OUTPUT|ENERPRO> [<ch>]
!EVT <tick_ms> FAULT CLEAR
!EVT <tick_ms> TRIGGER FIRING

```

- <tick_ms> is **plain decimal** milliseconds since boot (not zero-padded).
- <ch> is **1-based** and present **only** for OVERCURRENT, ENABLE_OUTPUT, and ENERPRO (the per-channel fault types).
- FAULT CLEAR has no channel.
- TRIGGER FIRING means the external PF15 trigger fired a shot.

Real examples:

```
!EVT 12345678 STATE ARMED
!EVT 12345679 STATE FIRING
!EVT 12345999 FAULT OVERCURRENT 1
!EVT 12346012 FAULT CLEAR
!EVT 12348000 TRIGGER FIRING
```

6.3 6.3 How to decode in LabVIEW

1. 1

2. 2

- starts with OK → it's a command reply (route to the request queue);
- starts with ERR → it's an error reply (parse <code>);
- starts with !EVT → it's a telemetry event.

3. 3

- token[0] = !EVT
- token[1] = tick_ms (U32)
- token[2] = event kind: STATE / FAULT / TRIGGER
- token[3] = payload (state name, fault type, or CLEAR/FIRING)
- token[4] (optional) = channel (U8, 1-based), only for the per-channel fault types.

4. 4

- STATE <name> → drive the state indicator.
- FAULT <type> [<ch>] → latch the fault lamp + show the fault type/channel.
- FAULT CLEAR → clear the fault lamp.
- TRIGGER FIRING → log/indicate an external-triggered shot.

6.4 6.4 Reading the flight recorder after a fault

SYS:EVLOG? returns OK <n> followed by exactly n one-line !EVT packets (oldest first). Read the OK <n> header, then read n lines and decode each per §6.3. This is the post-mortem event history and survives SYS:EVENT 0.

7 Appendix A — key numeric limits (this build)

Constant	Value
Channel count N	4
Output frequency range	3000 – 150000 Hz
PID loop rate	1000 Hz
Demand current range	0 – 6000 A per channel
Turn-on frequency (0 A)	5000 Hz
Max carrier frequency	100000 Hz
Fault ramp-down time	1.0 s
Waveform log capacity	1750 samples/channel
Nickname max length	15 chars